

MLForge — Distributed ML Model Serving & Monitoring Platform

Master Engineering Specification & Implementation Guide

1. PROJECT GOAL

Build a **production-style, locally deployable distributed machine-learning platform** that demonstrates real software-engineering principles rather than simply demonstrating an ML model.

The platform must allow a user to:

1. Register/login.
2. Upload a dataset.
3. Create a machine-learning training job.
4. Submit the training job asynchronously.
5. Track its status.
6. Train and evaluate a model.
7. Register/version the trained model.
8. Deploy/select a model version for inference.
9. Send prediction requests through an API.
10. Cache frequently requested predictions.
11. Monitor services and inference performance.
12. Detect failures.
13. Retry transient failures safely.
14. Prevent duplicate processing through idempotency.
15. Recover from worker/service failures.
16. Scale prediction services horizontally.
17. Run automated tests through CI/CD.
18. Observe system behavior through Prometheus and Grafana.

The project should demonstrate:

- Microservices
- Distributed systems
- REST APIs
- Asynchronous messaging
- Message queues
- Independent service responsibilities
- Database-backed persistence
- Caching
- Docker/containerization

- CI/CD
- Testing
- Retries
- Idempotency
- Fault tolerance
- Failure recovery
- Scalability
- Load testing
- Monitoring
- Logging
- Debugging
- Troubleshooting
- Operational excellence
- SDLC

2. IMPORTANT PROJECT CONSTRAINT

The entire project must work **locally without paid cloud infrastructure**.

Do NOT require:

- AWS EC2
- AWS ECS
- AWS EKS
- AWS RDS
- AWS Lambda
- AWS SQS
- paid APIs
- paid ML inference services

The architecture should be **cloud-ready**, but the actual implementation should run through Docker Compose on a local machine.

Do not falsely describe this project as an AWS deployment.

The AWS experience on the resume comes from the separate Self-Healing project.

3. CORE ENGINEERING PRINCIPLE

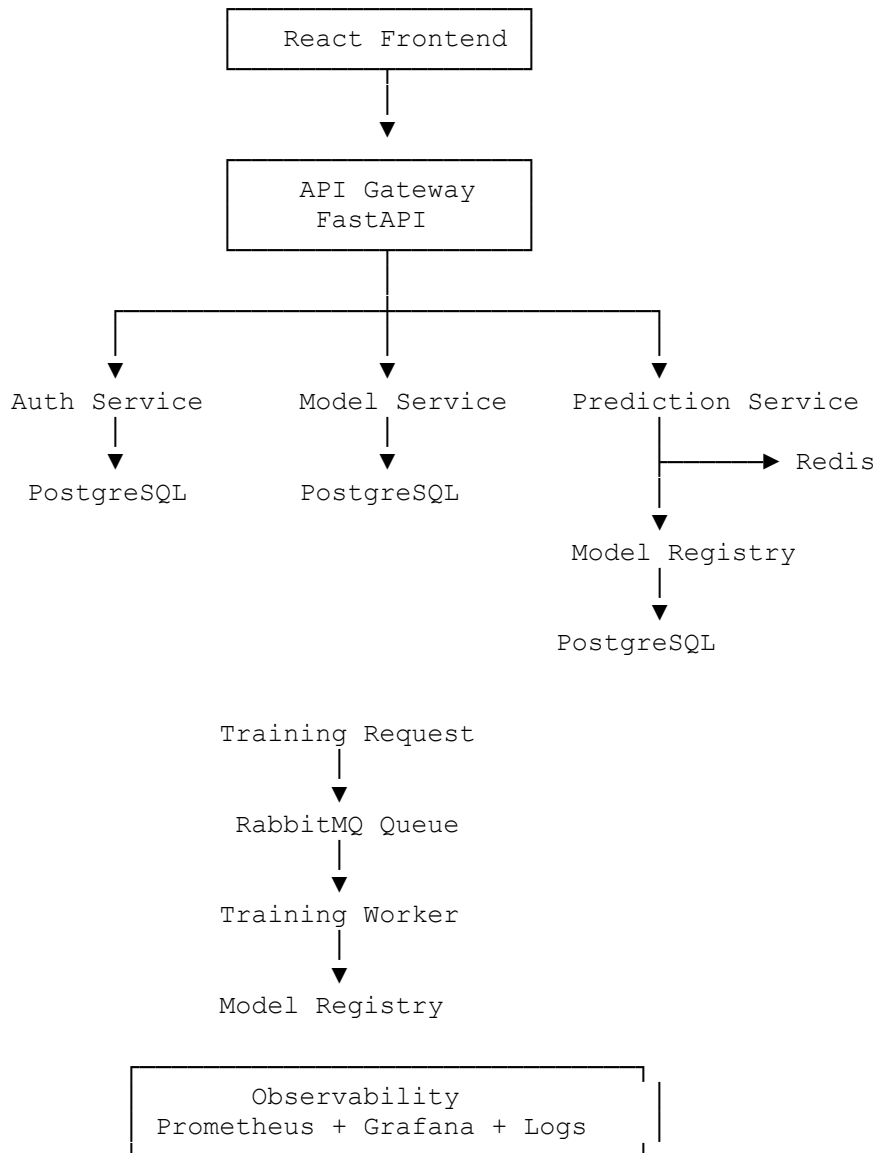
Do NOT build this as:

One FastAPI application

↓
One ML model
↓
One database

That would not demonstrate the required engineering concepts.

Instead build:



Each service must have a clear responsibility.

4. TECHNOLOGY STACK

Use the following stack unless there is a strong technical reason not to.

Backend

- Python 3.12+
- FastAPI
- Pydantic
- Uvicorn

Machine Learning

- scikit-learn
- pandas
- NumPy
- joblib

Use simple, explainable models.

Suggested models:

- Logistic Regression
- Random Forest
- Gradient Boosting

Do not spend excessive time making the ML sophisticated.

The distributed architecture is the main purpose of this project.

Database

PostgreSQL

Cache

Redis

Message Queue

RabbitMQ

Containerization

Docker

Docker Compose

Reverse Proxy / Load Balancing

Nginx

Monitoring

Prometheus
Grafana

Testing

pytest
pytest-asyncio
httpx

Load Testing

Locust

CI/CD

GitHub Actions

Version Control

Git + GitHub

Frontend

React or Next.js.

Keep the frontend simple.

The backend architecture is more important than UI.

5. SERVICES

Create the following services.

Service 1 — API Gateway

Responsibility:

- single public entry point
- request routing
- authentication forwarding/validation
- correlation/request ID creation

- basic request logging
- rate limiting if practical

The frontend should NOT directly communicate with every microservice.

Instead:

```
Frontend
  ↓
API Gateway
  ↓
Internal services
```

Example routes:

```
/api/auth/*
/api/models/*
/api/training/*
/api/predictions/*
```

6. AUTH SERVICE

Responsibility:

- user registration
- login
- authentication
- password hashing
- JWT generation
- user identity validation

Endpoints:

```
POST /auth/register
POST /auth/login
GET /auth/me
GET /health
GET /ready
```

Database table:

```
users
-----
id
email
password_hash
created_at
updated_at
```

Do not store plaintext passwords.

Use a secure password hashing library.

7. MODEL SERVICE

Responsibility:

- model metadata
- model versions
- model registration
- model status
- model selection
- model lifecycle

Endpoints:

```
POST /models
GET  /models
GET  /models/{model_id}
POST /models/{model_id}/versions
GET  /models/{model_id}/versions
GET  /models/{model_id}/versions/{version_id}
POST /models/{model_id}/versions/{version_id}/activate
GET  /health
GET  /ready
```

Database structure:

```
models
-----
id
name
description
created_by
created_at

model_versions
-----
id
model_id
version
algorithm
artifact_path
metrics_json
status
created_at
```

Possible status values:

```
TRAINING
READY
ACTIVE
FAILED
ARCHIVED
```

8. TRAINING SERVICE

This service accepts training requests.

Important:

Training should NOT block the HTTP request.

Bad architecture:

```
POST /train
  ↓
HTTP waits 5 minutes
  ↓
model finishes
  ↓
response
```

Correct architecture:

```
POST /train
  ↓
Create job
  ↓
Publish message
  ↓
Return immediately
  ↓
HTTP 202 Accepted
```

Example response:

```
{
  "job_id": "uuid",
  "status": "QUEUED"
}
```

The training job is then handled asynchronously.

9. RABBITMQ MESSAGE FLOW

Use RabbitMQ for asynchronous communication.

Flow:

```
Training Service
  ↓
RabbitMQ Exchange
  ↓
Training Queue
  ↓
```


Training Worker

Suggested message:

```
{
  "event_id": "uuid",
  "job_id": "uuid",
  "model_id": "uuid",
  "dataset_path": "/data/example.csv",
  "algorithm": "random_forest",
  "requested_at": "timestamp"
}
```

Every event must contain a unique:

event_id

Every training job must have a:

job_id

10. TRAINING WORKER

The worker consumes RabbitMQ messages.

Flow:

```
Receive message
  ↓
Validate message
  ↓
Check idempotency
  ↓
Mark job RUNNING
  ↓
Load dataset
  ↓
Preprocess data
  ↓
Train model
  ↓
Evaluate model
  ↓
Save model artifact
  ↓
Register model version
  ↓
Mark job COMPLETED
  ↓
Acknowledge RabbitMQ message
```

If training fails:

Training failure

↓
Record error
↓
Increment retry count
↓
Retry if transient
↓
Dead-letter if retries exhausted

11. RETRY POLICY

Retries must NOT be infinite.

Implement:

Maximum retries = 3

Use exponential backoff.

Example:

Attempt 1 → immediate
Attempt 2 → 2 seconds
Attempt 3 → 4 seconds
Attempt 4 → 8 seconds

Do not retry permanent errors.

Example:

Retry

- temporary database connection failure
- temporary RabbitMQ connection failure
- transient network error

Do NOT retry

- invalid dataset
- missing required column
- unsupported algorithm
- malformed request

12. IDEMPOTENCY

This is mandatory.

A distributed system may deliver the same message more than once.

Example:

```
Worker receives event A
      ↓
Training starts
      ↓
Worker crashes before ACK
      ↓
RabbitMQ redelivers event A
      ↓
Worker receives event A again
```

Without idempotency:

```
same training job
      ↓
trained twice
      ↓
duplicate model versions
```

Implement an idempotency mechanism.

Use a database table:

```
processed_events
-----
event_id
processed_at
status
```

Before processing:

```
Does event_id already exist?

YES → do not process again
NO  → process
```

The database operation must be atomic enough to prevent two workers from simultaneously processing the same event.

Use a unique constraint on:

```
event_id
```

This is important.

13. DEAD-LETTER QUEUE

Configure a RabbitMQ dead-letter queue.

Flow:

```
Training Queue
  ↓
Consumer fails
  ↓
Retry 1
  ↓
Retry 2
  ↓
Retry 3
  ↓
Still failing
  ↓
Dead Letter Queue
```

Create a separate queue:

```
training.dead
```

Do not silently discard failed messages.

The system must preserve failed work for investigation.

14. PREDICTION SERVICE

Responsibility:

- load active model
- validate input
- generate predictions
- return prediction response
- expose metrics
- handle failures

Endpoint:

```
POST /predictions
GET  /predictions/{request_id}
GET  /health
GET  /ready
```

Example request:

```
{
  "model_id": "equipment-failure",
  "model_version": "v3",
  "features": {
    "temperature": 72.4,
    "pressure": 10.2,
    "vibration": 0.84
  }
}
```

```
}
```

Response:

```
{
  "request_id": "uuid",
  "model_id": "equipment-failure",
  "model_version": "v3",
  "prediction": 1,
  "confidence": 0.94,
  "latency_ms": 18
}
```

15. REDIS CACHING

Use Redis for prediction caching.

Cache key:

```
prediction:{model_version}:{hash(features)}
```

Flow:

```
Prediction Request
↓
Generate deterministic cache key
↓
Check Redis
├── HIT → return cached prediction
└── MISS
    ↓
    Load model
    ↓
    Generate prediction
    ↓
    Save result in Redis
    ↓
    Return response
```

Use a reasonable TTL.

Example:

```
TTL = 5 minutes
```

Do not cache requests that contain nondeterministic behavior.

16. DATABASE RESPONSIBILITY

PostgreSQL should persist important system state.

Minimum tables:

```
users
models
model_versions
training_jobs
prediction_requests
processed_events
```

Example:

```
training_jobs
-----
id
model_id
status
algorithm
retry_count
error_message
created_at
started_at
completed_at
```

Example:

```
prediction_requests
-----
id
model_id
model_version
input_hash
prediction
confidence
latency_ms
created_at
```

17. DATABASE DESIGN PRINCIPLE

Do not create one giant table.

Each table should represent a clear domain.

Use foreign keys where appropriate.

Example:

```
models
├── model_versions
│   └── prediction_requests
```

Indexes should be added for frequently queried fields.

For example:

```
training_jobs.status
training_jobs.created_at
prediction_requests.model_id
prediction_requests.created_at
```

18. MODEL ARTIFACT STORAGE

For the local project, store model artifacts on a mounted Docker volume.

Example:

```
./storage/models/
```

Do not upload model binaries into Git.

Add:

```
storage/
*.joblib
*.pkl
```

to `.gitignore`.

The database should store the artifact path and metadata.

19. API COMMUNICATION

Use REST for synchronous operations.

Example:

```
Frontend
  ↓ REST
API Gateway
  ↓ REST
Model Service
```

Use RabbitMQ for operations that should be asynchronous.

Example:

```
Training API
  ↓
RabbitMQ
```

↓
Training Worker

This distinction must be clearly documented.

20. REQUEST IDs / CORRELATION IDs

Every incoming request must receive a unique:

`request_id`

Example:

`req_7f2c91`

Propagate it between services.

Example:

```
Gateway
request_id=req_7f2c91
  ↓
Prediction Service
request_id=req_7f2c91
  ↓
Model Service
request_id=req_7f2c91
```

This is essential for distributed debugging.

21. LOGGING

Every service must use structured logs.

Each log should include:

```
timestamp
service
level
request_id
event
message
duration_ms
```

Example:

```
2026-08-09T10:20:31Z
service=prediction-service
```



```
level=INFO
request_id=req_7f2c91
event=prediction_completed
duration_ms=18
```

Use:

```
INFO
WARNING
ERROR
```

appropriately.

Do not log passwords, tokens, or sensitive data.

22. HEALTH ENDPOINTS

Every service must expose:

```
GET /health
GET /ready
```

/health

Checks whether the process is alive.

/ready

Checks whether the service can actually serve traffic.

For example, prediction service readiness may check:

```
PostgreSQL reachable
Redis reachable
model registry available
```

This distinction should be documented.

23. DOCKER ARCHITECTURE

Every service must have its own Dockerfile.

Example:

```
services/
├── api-gateway/
│   └── Dockerfile
```

```
|— auth-service/  
  |— Dockerfile  
|— model-service/  
  |— Dockerfile  
|— training-service/  
  |— Dockerfile  
|— training-worker/  
  |— Dockerfile  
|— prediction-service/  
  |— Dockerfile
```

Infrastructure containers:

```
postgres  
redis  
rabbitmq  
prometheus  
grafana  
nginx
```

Everything should start through:

```
docker compose up --build
```

The entire system must be reproducible from a clean machine.

24. DOCKER NETWORKING

All services should communicate using Docker Compose service names.

Example:

```
postgres:5432  
redis:6379  
rabbitmq:5672  
prediction-service:8000
```

Do NOT hardcode:

```
localhost
```

for internal service-to-service communication.

This is important because `localhost` inside a container refers to that container itself.

25. ENVIRONMENT CONFIGURATION

Do not hardcode credentials.

Use:

```
.env
```

Example:

```
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=mlforge
POSTGRES_USER=mlforge
POSTGRES_PASSWORD=...
REDIS_HOST=redis
RABBITMQ_HOST=rabbitmq
```

Commit:

```
.env.example
```

Do NOT commit:

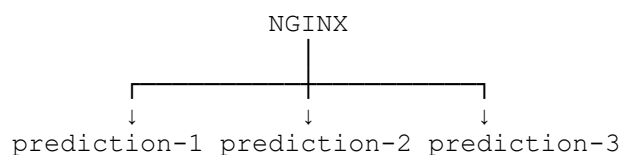
```
.env
```

26. NGINX LOAD BALANCING

The prediction service must be horizontally scalable.

Run multiple prediction-service containers.

Example:



Nginx distributes incoming requests.

Use Docker Compose scaling or explicitly defined replicas.

The system must demonstrate that the prediction service is stateless enough to run multiple instances.

Shared state should be outside the container:

- PostgreSQL
 - Redis
 - model artifact volume/shared storage
-

27. SCALABILITY TEST

Use Locust.

Create a load-test scenario.

Example:

```
10 users
50 users
100 users
```

Measure:

- requests/sec
- average latency
- p50 latency
- p95 latency
- p99 latency
- error rate

Record results.

Do not invent results.

The final README must contain the actual measured results.

28. SCALABILITY EXPERIMENT

Perform at least two experiments.

Experiment A

One prediction service instance.

Record:

```
throughput
latency
error rate
```

Experiment B

Three prediction service instances behind Nginx.

Record the same metrics.

Compare:

1 instance vs 3 instances

The purpose is to demonstrate actual horizontal scaling.

29. FAILURE SCENARIO 1 — PREDICTION SERVICE CRASH

Start the platform.

Send requests.

Then intentionally stop one prediction service container.

Example:

```
docker stop prediction-service-1
```

Expected behavior:

```
Prediction service crashes
      ↓
Nginx no longer sends traffic to it
      ↓
Other instances continue processing
      ↓
System remains available
```

Document:

- what failed
 - how it was detected
 - what happened to requests
 - whether errors increased
 - how recovery happened
-

30. FAILURE SCENARIO 2 — TRAINING WORKER CRASH

Start a training job.

Force the worker to stop during processing.

Expected:

```
Worker crashes
  ↓
RabbitMQ does not receive ACK
  ↓
Message remains/reappears
  ↓
Worker restarts
  ↓
Message is redelivered
  ↓
Idempotency check occurs
  ↓
Job is safely processed
```

This is one of the most important distributed-system demonstrations.

31. FAILURE SCENARIO 3 — DATABASE TEMPORARILY UNAVAILABLE

Stop PostgreSQL.

Send a request.

Expected:

```
Service detects DB failure
  ↓
Request fails gracefully
  ↓
Error logged
  ↓
Metrics updated
  ↓
Database restarted
  ↓
Service becomes ready again
```

Do not allow the service to crash uncontrollably.

32. FAILURE SCENARIO 4 — REDIS FAILURE

Stop Redis.

The prediction service should not completely collapse simply because the cache is unavailable.

Preferred behavior:

```
Redis unavailable
  ↓
Log warning
  ↓
Skip cache
  ↓
Perform prediction normally
```

This demonstrates graceful degradation.

33. FAILURE SCENARIO 5 — INVALID TRAINING DATA

Submit a malformed dataset.

Expected:

```
Validation fails
  ↓
Training job marked FAILED
  ↓
Useful error recorded
  ↓
No infinite retry
  ↓
No corrupted model registered
```

This demonstrates the difference between:

transient failure

and:

permanent failure

34. MONITORING WITH PROMETHEUS

Every relevant service must expose:

```
GET /metrics
```

Collect at minimum:

HTTP metrics

- request count
- request latency
- error count

Service metrics

- active requests
- process CPU
- memory

ML metrics

- prediction count
- prediction latency
- model version usage

Queue metrics

- messages processed
- messages failed
- retry count
- dead-letter count

35. GRAFANA DASHBOARD

Create one main dashboard.

Sections:

System Overview

Show:

Requests/sec
Error rate
p95 latency
Active services

Prediction Service

Show:

prediction requests
prediction latency
prediction errors
model version distribution

RabbitMQ

Show:

queue depth
messages processed
retry count
dead-letter messages

Infrastructure

Show:

CPU
memory
container health

36. ALERTING

Create practical thresholds.

Examples:

Error rate > 5%
p95 latency > 500 ms
Queue depth > 100
Dead-letter messages > 0
Service unavailable

The goal is to demonstrate that operators can identify a problem.

37. DEBUGGING WORKFLOW

Create a documented troubleshooting process.

When something fails:

1. Identify symptom
2. Check Grafana
3. Check service health
4. Search logs using request_id
5. Identify failing dependency
6. Reproduce locally

7. Inspect exception/metrics
8. Determine root cause
9. Implement fix
10. Add regression test
11. Re-run load/integration tests
12. Document resolution

Do not simply say:

"Fixed bug."

Document actual incidents.

38. INCIDENT REPORTS

Create:

docs/incidents/

Include at least 3 incident reports.

Example:

INC-001-prediction-service-failure.md
INC-002-worker-crash.md
INC-003-redis-outage.md

Each report should contain:

Incident
Date
Severity
Affected Service
Symptoms
Detection
Timeline
Root Cause
Impact
Resolution
Preventive Action
Regression Test

This is extremely valuable for interviews.

39. CI PIPELINE

Create:

`.github/workflows/ci.yml`

Every push and pull request should run:

```
Checkout
↓
Set up Python
↓
Install dependencies
↓
Lint
↓
Unit tests
↓
Integration tests
↓
Build Docker images
```

Use GitHub Actions.

40. CI QUALITY GATES

The pipeline should fail if:

- tests fail
- linting fails
- Docker build fails

Do not allow broken code to reach main.

41. CD PIPELINE

Create:

`.github/workflows/cd.yml`

After successful CI:

```
Build Docker images
↓
Tag image
↓
Push image to GitHub Container Registry
```

This is enough for a local/cloud-ready project.

Do not claim cloud deployment.

42. GITHUB WORKFLOW

Use:

```
main
develop
feature/*
```

Example:

```
feature/prediction-service
feature/rabbitmq-worker
feature/monitoring
feature/load-testing
```

Do not develop everything directly on main.

43. PULL REQUEST WORKFLOW

For every meaningful feature:

```
Create branch
↓
Implement
↓
Write tests
↓
Push
↓
Create Pull Request
↓
CI runs
↓
Review
↓
Fix review comments
↓
Merge
```

If working alone, do not falsely claim another engineer reviewed the code.

Use GitHub's PR workflow genuinely.

If possible, have a classmate review at least one or two PRs.

44. TESTING STRATEGY

Implement three levels.

Unit tests

Test:

- feature validation
- model prediction
- retry calculation
- idempotency logic
- cache key generation
- request validation

Integration tests

Test:

API → PostgreSQL
API → Redis
Training Service → RabbitMQ → Worker
Prediction Service → Model Registry

End-to-end test

Test:

Login
↓
Create model
↓
Submit training job
↓
RabbitMQ
↓
Worker
↓
Model registered
↓
Prediction request
↓
Prediction returned

45. IDEMPOTENCY TEST

Write a specific test:

Send same event_id twice

Expected:

First request → processed
Second request → ignored/deduplicated

Verify that only one training job/model version is created.

This test is mandatory.

46. RETRY TEST

Simulate a transient failure.

Example:

```
Worker attempts database update
      ↓
temporary failure
      ↓
retry
      ↓
successful operation
```

Verify:

retry_count increments

and the system eventually succeeds.

47. DEAD-LETTER TEST

Create a deliberately failing message.

Allow retries to exhaust.

Expected:

```
Training Queue
      ↓
Retry
      ↓
Retry
      ↓
Retry
      ↓
Dead Letter Queue
```

Verify the message exists in the dead-letter queue.

48. CACHE TEST

Perform the same prediction twice.

First request:

```
Redis MISS
↓
model inference
↓
Redis SET
```

Second request:

```
Redis HIT
↓
cached response
```

Measure latency difference if practical.

49. HEALTH CHECK TEST

Stop PostgreSQL.

Then:

```
GET /ready
```

should report the service as not ready.

Restart PostgreSQL.

Then:

```
GET /ready
```

should return healthy again.

50. API DOCUMENTATION

FastAPI automatically generates:

/docs
/redoc

Every endpoint must have:

- summary
- description
- request schema
- response schema
- status codes
- error behavior

Do not leave API documentation generic.

51. ERROR HANDLING

Use consistent error responses.

Example:

```
{  
  "error": "MODEL_NOT_FOUND",  
  "message": "Requested model version does not exist",  
  "request_id": "req_123"  
}
```

Use appropriate HTTP status codes.

Examples:

```
400 → validation error  
401 → unauthorized  
404 → not found  
409 → conflict/idempotency  
422 → invalid request  
429 → rate limit  
500 → unexpected server error  
503 → dependency unavailable
```

52. GRACEFUL SHUTDOWN

Services should handle termination signals.

When a worker shuts down:

```
Stop accepting new work  
  ↓  
Finish current safe operation
```



```
↓
Close DB connections
↓
Close Redis connection
↓
Close RabbitMQ connection
↓
Exit
```

Do not abruptly terminate active work when avoidable.

53. DATABASE CONNECTION MANAGEMENT

Use connection pooling.

Do not create a new database connection for every request if avoidable.

Configure:

- pool size
- timeout
- connection recycling

Handle connection failures gracefully.

54. SECURITY BASICS

Implement:

- password hashing
- JWT authentication
- environment-based secrets
- no committed credentials
- input validation
- parameterized SQL / ORM
- CORS configuration
- basic rate limiting

Do not overengineer security.

This is a demonstration project.

55. REPOSITORY STRUCTURE

Use this structure:

```
mlforge/
├── services/
│   ├── api-gateway/
│   │   ├── app/
│   │   ├── tests/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── auth-service/
│   │   ├── app/
│   │   ├── tests/
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── model-service/
│   ├── training-service/
│   ├── training-worker/
│   └── prediction-service/
├── frontend/
├── infrastructure/
│   ├── nginx/
│   ├── prometheus/
│   └── grafana/
├── tests/
│   ├── integration/
│   └── e2e/
├── load-tests/
│   └── locustfile.py
├── docs/
│   ├── architecture.md
│   ├── api.md
│   ├── failure-handling.md
│   ├── scalability.md
│   ├── debugging.md
│   └── incidents/
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── cd.yml
├── storage/
│   └── models/
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
└── CONTRIBUTING.md
```

56. IMPLEMENTATION ORDER

Do NOT try to build everything simultaneously.

Follow this exact order.

PHASE 1 — Repository setup

Create:

```
Git repository
README
.gitignore
.env.example
Docker Compose
basic documentation
```

Goal:

All developers/tools know the project structure.

PHASE 2 — PostgreSQL

Create:

```
users
models
model_versions
training_jobs
prediction_requests
processed_events
```

Goal:

Persistent data layer works.

PHASE 3 — Auth Service

Implement:

```
register
login
JWT
/me
```

health
ready

Goal:

Authentication works independently.

PHASE 4 — Model Service

Implement:

model creation
model listing
version management
activation

Goal:

Model metadata can be persisted and retrieved.

PHASE 5 — RabbitMQ

Create:

exchange
training queue
dead-letter queue

Goal:

Messages can move reliably between training service and worker.

PHASE 6 — Training Service

Implement:

POST /training/jobs
GET /training/jobs/{job_id}

Submitting a job should immediately return:

202 Accepted

Goal:

Training becomes asynchronous.

PHASE 7 — Training Worker

Implement:

```
consumer
model training
evaluation
artifact persistence
model registration
ACK
retry
idempotency
dead-letter handling
```

Goal:

A complete asynchronous training pipeline works.

PHASE 8 — Prediction Service

Implement:

```
POST /predictions
```

Goal:

Registered model can generate predictions.

PHASE 9 — Redis

Implement:

```
cache lookup
cache write
TTL
graceful cache failure
```

Goal:

Repeated predictions can be served from cache.

PHASE 10 — API Gateway

Route:

```
/auth  
/models  
/training  
/predictions
```

Goal:

Frontend communicates through a single entry point.

PHASE 11 — Dockerize EVERYTHING

Every service must run inside Docker.

Goal:

The entire platform starts using:

```
docker compose up --build
```

PHASE 12 — Nginx

Run multiple prediction service instances.

Goal:

Demonstrate horizontal scaling.

PHASE 13 — Monitoring

Add:

```
Prometheus  
Grafana  
/metrics  
structured logs  
request IDs
```

Goal:

System becomes observable.

PHASE 14 — CI/CD

Implement:

```
GitHub Actions
lint
unit tests
integration tests
Docker builds
image publishing
```

Goal:

Every change is automatically validated.

PHASE 15 — Load Testing

Use Locust.

Test:

```
10 users
50 users
100 users
```

Compare:

```
1 prediction instance
vs
3 prediction instances
```

Goal:

Produce actual scalability measurements.

PHASE 16 — FAILURE TESTING

Intentionally cause:

```
prediction service crash
worker crash
PostgreSQL outage
Redis outage
invalid training data
RabbitMQ failure
```

Goal:

Prove fault tolerance and recovery.

PHASE 17 — Documentation

Document:

- architecture
- API
- database schema
- queue design
- retry strategy
- idempotency
- failure scenarios
- monitoring
- scalability
- debugging
- incident reports

Goal:

Someone else should be able to understand the entire system without asking you questions.

57. DEFINITION OF DONE

The project is NOT complete merely because the UI works.

MLForge is complete only when all of these are true:

```
[ ] Multiple independent services exist
[ ] Services have clear responsibilities
[ ] REST communication works
[ ] RabbitMQ asynchronous communication works
[ ] PostgreSQL persistence works
[ ] Redis caching works
[ ] Docker Compose starts the system
[ ] Model training is asynchronous
[ ] Retry mechanism works
[ ] Idempotency works
[ ] Dead-letter queue works
[ ] Prediction service can scale horizontally
[ ] Nginx distributes traffic
[ ] Prometheus collects metrics
[ ] Grafana dashboard works
[ ] Structured logging works
[ ] Correlation/request IDs work
[ ] Health endpoints work
```


- [] CI pipeline works
- [] Docker images build automatically
- [] Unit tests exist
- [] Integration tests exist
- [] E2E workflow works
- [] Locust load testing works
- [] Failure scenarios have been tested
- [] Incident reports exist
- [] README explains setup
- [] API documentation exists
- [] No secrets are committed
- [] Git history demonstrates feature branches/PR workflow

58. REQUIRED README STRUCTURE

The README must contain:

1. Project Overview
 2. Problem Statement
 3. Goals
 4. Architecture
 5. Service Responsibilities
 6. Technology Stack
 7. Request Flow
 8. Asynchronous Training Flow
 9. Database Design
 10. RabbitMQ Design
 11. Retry Strategy
 12. Idempotency Strategy
 13. Fault Tolerance
 14. Caching Strategy
 15. Scalability
 16. Monitoring
 17. CI/CD
 18. Testing
 19. Failure Scenarios
 20. Debugging
 21. Installation
 22. Running Locally
 23. API Documentation
 24. Load Test Results
 25. Known Limitations
 26. Future Improvements
-

59. REQUIRED ARCHITECTURE DIAGRAMS

Create at least these diagrams.

Diagram 1 — High-Level Architecture

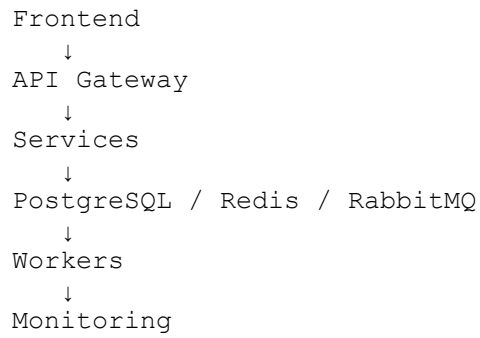


Diagram 2 — Training Flow

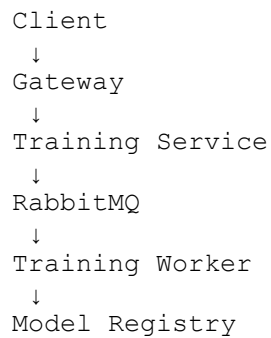


Diagram 3 — Prediction Flow

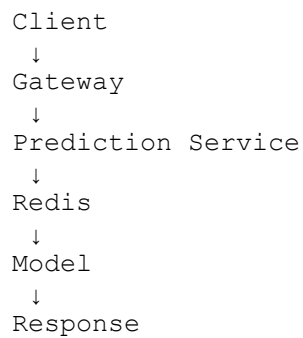


Diagram 4 — Failure Recovery

Show:

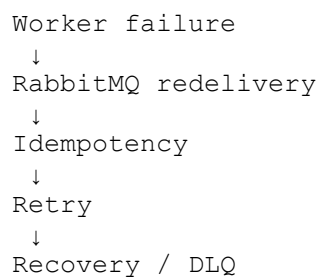


Diagram 5 — Horizontal Scaling

Show:

Nginx

- └─ Prediction 1
- └─ Prediction 2
- └─ Prediction 3

60. FINAL INTERVIEW-LEVEL EXPLANATION

Once implemented, the project should be explainable in this sequence:

Problem

Traditional ML prototypes often combine training, prediction, data persistence, and model management into one application. That makes the system difficult to scale, monitor, debug, and recover when individual components fail.

Solution

MLForge separates these responsibilities into independently deployable services.

Architecture

The API Gateway provides one entry point.

The model service manages model metadata and versions.

The training service accepts training requests and publishes asynchronous jobs to RabbitMQ.

Training workers consume those jobs and train models independently.

The prediction service handles real-time inference.

PostgreSQL stores durable application state.

Redis provides low-latency caching.

Prometheus and Grafana provide observability.

Docker packages every component consistently.

Nginx allows prediction services to scale horizontally.

GitHub Actions automates testing and image builds.

Distributed-system design

The most important asynchronous workflow is:

```
Training Request
  ↓
Training Service
  ↓
RabbitMQ
  ↓
Training Worker
```

This prevents long-running training jobs from blocking API requests.

Reliability

Retries handle transient failures.

Idempotency prevents duplicate message processing.

Dead-letter queues preserve messages that cannot be successfully processed.

Health/readiness checks make service health observable.

Graceful degradation allows prediction to continue if Redis becomes unavailable.

Scalability

Prediction services are stateless enough to run multiple replicas.

Nginx distributes traffic across replicas.

Locust measures performance under increasing concurrent load.

Observability

Prometheus collects metrics.

Grafana visualizes:

- latency
- throughput
- errors
- queue depth
- retries
- CPU
- memory
- prediction performance

Correlation IDs allow a request to be traced across services.

Debugging

When something fails:

```
Metric/alert
↓
Request ID
↓
Logs
↓
Service health
↓
Dependency status
↓
Root cause
↓
Fix
↓
Regression test
```

CI/CD

Every code change goes through:

```
Git push
↓
GitHub Actions
↓
Lint
↓
Unit tests
↓
Integration tests
↓
Docker build
↓
Image publishing
```

This provides a repeatable software delivery process.

61. MOST IMPORTANT RULE FOR ANTIGRAVITY

Do not optimize for completing the project quickly by removing the engineering concepts.

Do not create fake implementations that only print:

```
retry successful
service recovered
```

The behaviors must be real.

For example:

If the worker crashes before acknowledging a RabbitMQ message, RabbitMQ should genuinely redeliver the message.

If the same event is delivered twice, the database idempotency constraint should genuinely prevent duplicate processing.

If Redis goes down, the prediction service should genuinely degrade gracefully.

If one prediction container is stopped, the other replicas should genuinely continue serving requests.

If CI tests fail, GitHub Actions should genuinely fail the workflow.

If the database is unavailable, readiness should genuinely fail.

The project must demonstrate **actual engineering behavior**, not simulated output.

62. ANTIGRAVITY EXECUTION INSTRUCTION

Build MLForge incrementally according to the phases above.

For every phase:

1. Implement the feature.
2. Run it.
3. Test it.
4. Fix errors.
5. Add automated tests.
6. Update documentation.
7. Do not proceed until the current phase works.
8. Keep the code production-style and readable.
9. Do not fabricate test results or performance metrics.
10. Do not mark a feature complete unless it has actually been implemented and tested.

At the end, produce:

1. Fully working source code
2. Docker Compose environment
3. GitHub Actions CI/CD
4. Prometheus configuration
5. Grafana dashboard
6. Locust load tests
7. Automated tests
8. Architecture diagrams
9. Incident reports
10. Complete README
11. API documentation

- 12. Actual benchmark results
- 13. Actual failure-test results

The final system must be reproducible from a clean clone using documented commands.

63. FINAL PROJECT OBJECTIVE

The finished MLForge project should demonstrate that the developer understands how to move from:

ML Prototype

to:

Distributed Software System

and can reason about:

Architecture
↓
Services
↓
Communication
↓
Persistence
↓
Asynchronous Processing
↓
Retries
↓
Idempotency
↓
Fault Tolerance
↓
Scalability
↓
Monitoring
↓
Debugging
↓
Testing
↓
CI/CD
↓
Operational Excellence

The ML model is only one component.

The actual project is the distributed engineering system around it.